

Suite du Guide Technique : Modélisation & Envoi Multi-Canal

Modèles ORM Django, Intégration Firebase Cloud Messaging & Configuration Celery Beat

Étape 5 : Schéma de Modèles PostgreSQL Complète (models.py)

Afin de gérer les transactions, les préférences de filtres et les notifications, voici l'architecture exacte des modèles de base de données à implémenter :

```
from django.db import models
from django.contrib.auth.models import AbstractUser

class User(AbstractUser):
    telephone = models.CharField(max_length=20, unique=True)
    preferences_concours = models.JSONField(default=dict, blank=True) # Ex: {"filieres": ["ENSP", "IDE"], "regions": ["Centre"]}

class Concours(models.Model):
    STATUT_CHOICES = [
        ('OUVERT', 'Ouvert'),
        ('MODIFIE', 'Modifié'),
        ('ANNULE', 'Annulé'),
        ('CLOTURE', 'Clôturé'),
    ]
    nom = models.CharField(max_length=255)
    filiere = models.CharField(max_length=100)
    date_limite = models.DateTimeField()
    statut = models.CharField(max_length=20, choices=STATUT_CHOICES, default='OUVERT')
    prix_acces = models.DecimalField(max_digits=10, decimal_places=2, default=0.00)

class Transaction(models.Model):
    PROVIDER_CHOICES = [('MTN', 'MTN Mobile Money'), ('ORANGE', 'Orange Money')]
    STATUT_TX = [('PENDING', 'En attente'), ('SUCCESS', 'Succès'), ('FAILED', 'Échec')]
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    concours = models.ForeignKey(Concours, on_delete=models.CASCADE)
    reference = models.CharField(max_length=100, unique=True)
    montant = models.DecimalField(max_digits=10, decimal_places=2)
    provider = models.CharField(max_length=10, choices=PROVIDER_CHOICES)
    statut = models.CharField(max_length=10, choices=STATUT_TX, default='PENDING')
    created_at = models.DateTimeField(auto_now_add=True)
```

Étape 6 : Worker Celery pour Envoi de Push Mobile (Firebase FCM)

Cette tâche s'exécute en tâche de fond pour expédier les notifications push vers l'application mobile sans ralentir le serveur API :

```
# tasks.py
from celery import shared_task
from firebase_admin import messaging
from .models import User

@shared_task
def envoyer_notification_push_async(user_id, titre, message_txt):
    try:
        user = User.objects.get(id=user_id)
        # Envoi via Firebase Cloud Messaging SDK
        message = messaging.Message(
            notification=messaging.Notification(title=titre, body=message_txt),
            token=user.fcm_token # Token FCM enregistré lors de la connexion mobile
        )
        response = messaging.send(message)
        return f"Push envoyé à {user.username} : {response}"
    except Exception as e:
        return f"Erreur lors de l'envoi Push : {str(e)}"
```

Étape 7 : Configuration du Planificateur Celery Beat (settings.py)

Pour exécuter automatiquement la tâche de vérification des concours chaque soir à minuit :

```
# settings.py
from celery.schedules import crontab

CELERY_BEAT_SCHEDULE = {
    'verifier-concours-expire-chaque-minuit': {
        'task': 'votre_app.tasks.verifier_et_cloturer_concours',
        'schedule': crontab(hour=0, minute=0), # Exécution automatique à 00h00
    },
}
```

Étape 8 : Réponse d'un Endpoint Filtré en JSON (Exemple de sortie DRF)

Lorsque le client mobile ou web effectue une requête de filtrage `GET /api/concours/?filier=ENSP&statut=OUVERT` :

```
{
  "count": 1,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 12,
      "nom": "Concours Polytechnique ENSPY 2026",
      "filier": "Génie Informatique",
      "date_limite": "2026-08-15T23:59:00Z",
      "statut": "OUVERT",
      "prix_acces": "2000.00"
    }
  ]
}
```